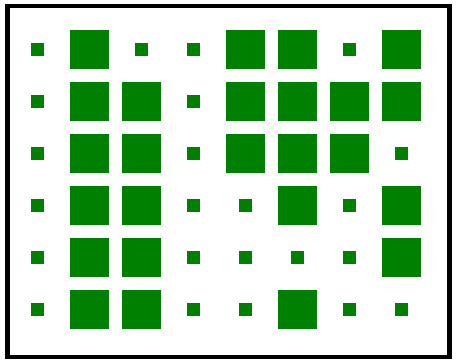




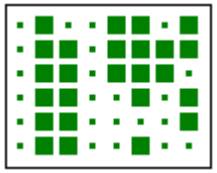
Class 3

Language Families, Compilation Overview

CS 330

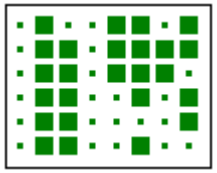
Programming Languages and Compilers

Categorizing Languages



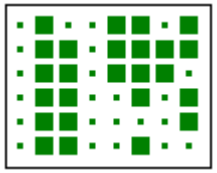
- Imperative
  - Written based on fetching ordered instructions, deciding, executing and then moving to the next (procedural, **von Neumann-centric**)
  - C, BASIC, FORTRAN, COBOL...
- Object Oriented (OO)
  - Focused on data and its relationship to functions (behaviors). Uses encapsulation, inheritance and polymorphism (dynamic method binding) to promote reliability
    - Extension to imperative (**von Neumann**)
  - Java, Python, R, C++, C#

Categorizing Languages

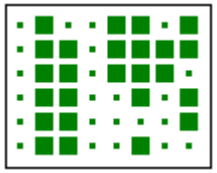


- Functional
  - Apply functions to parameters without reliance on variables, assignment statements, and iteration
  - Performance on von Neumann architectures tends to be slower than imperative and OO due to the disagreement between paradigms
  - Lisp, Clojure, Scheme, F#
- Declarative (Logic)
  - Declarative rules where order of operations is not explicitly defined by the programmer
  - Uses inferencing (forward or backward chaining) to produce a result
  - Prolog, OWL

A Key Language Takeaway

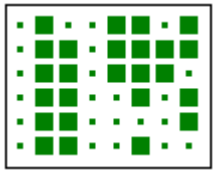


- In the general sense, there is no “natural” or “correct” paradigm for programming languages
  - A common computer architecture has caused many languages to be designed in a specific manner to align with its paradigms
    - Has habituated us to using loops, variables, procedures…
- There are other ways to approach describing a problem to a computer
  - If you step back and think about the problem you are trying to solve, you may find an alternate paradigm is a more natural fit



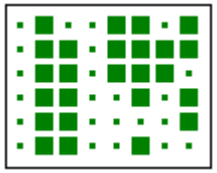
Why So Many Languages?

Trade-offs in Language Design



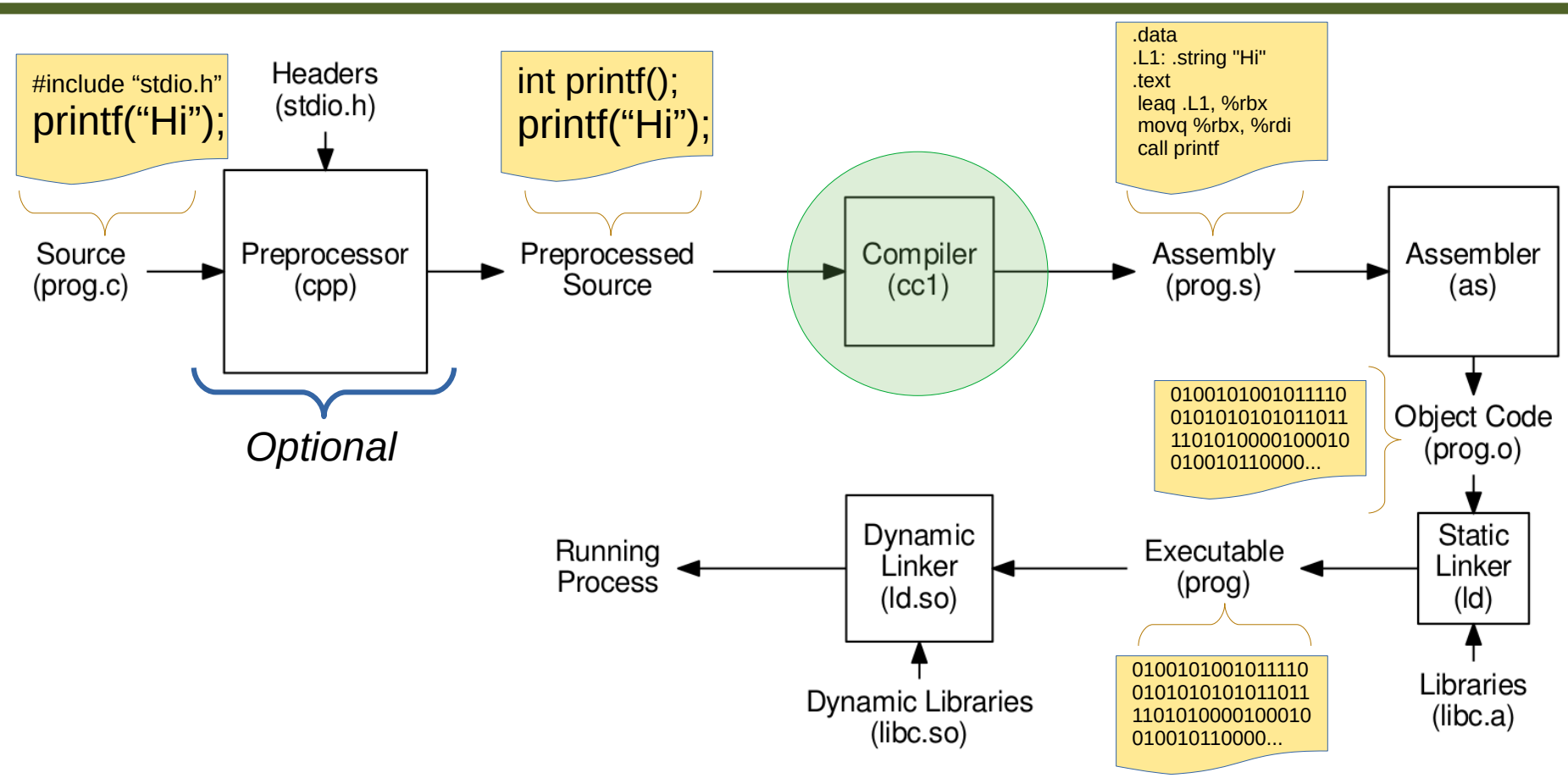
- Why not one language for each family?
  - Language designers make decisions based on readability, writeability, etc., that alter the dynamics of each language
- Depending on project goals, different drivers will influence language choice(s)
- Generally we trade readability/writeability and speed of execution for reliability

Interpretation vs. Compilation



- The computer's CPU uses a fixed instruction set
  - Instructions are fetched and executed
  - Instructions at this level are low-level operations that are shifting data between registers, carrying out basic mathematical operations, and moving data between CPU and memory
- High-level language programs are converted to use this fixed instruction set
  - On-the-fly (interpreted) or re-written to machine language (compiled)

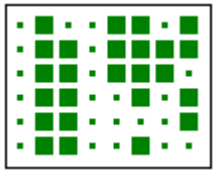
Compilation: Source to Executable



Thain, D. (2020). Introduction to compilers and language design.

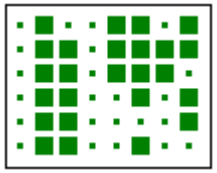
Figure 2.1: A Typical Compiler Toolchain

Preprocessor



- Some languages (C, C++) provide a specialized syntax that is used to manipulate source code in a scripted fashion
- Modifies the program code prior to its scanning by an interpreter or compiler
- Common capabilities of a preprocessor:
  - Allow inclusion of other program files
  - Permit optional inclusion of code during compilation
  - Define “macros” where one pattern replaces another
- A preprocessor is used to enhance readability, writeability, reliability, and flexibility of code

Compilation

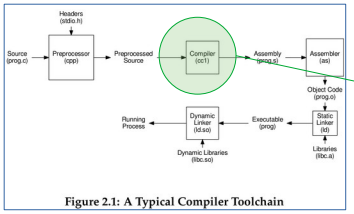


- The process of compilation involves:
  - Scanning the source code (tokens)
  - Parsing tokens (based on language grammar)
  - Building an Abstract Syntax Tree
  - (optional) optimization
  - Generating assembly code
  - Linking
  - Generating machine code

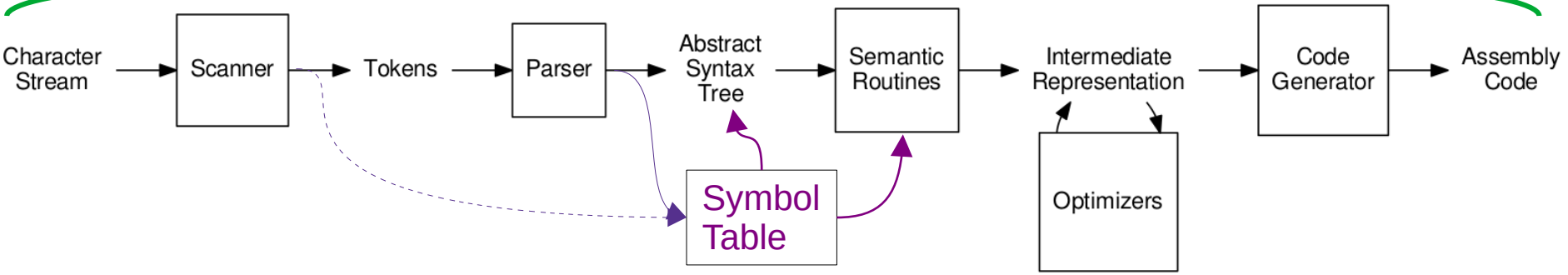
We are writing this

- The processing overhead (time, memory) of converting from programming language to machine code occurs ***before*** running the program. This happens once, and the program can be run multiple times

Compilation Process Overview

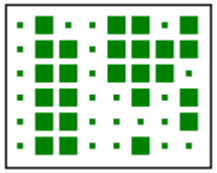


Thain, D. (2020). Introduction to compilers and language design.



- Scanner (Lexical analyzer) – tokenizes
- Parser (Syntax analyzer) – lexical units syntax check
  - Creates symbol table and AST
- Semantic Routines – static semantics analysis/check
- Intermediate Rep – lower level language (optional)
- Optimizer – performance improvements (optional)
- Code Generator – output assembly code (in our case)

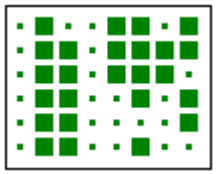
Tokenizing Example



```
height = (width+56) * factor(foo);
```

Token	Value
ID	height
ASSIGN	
OPEN_PAREN	
ID	width
ADD	
INT	56
CLOSE_PAREN	
MUL	
ID	factor
OPEN_PAREN	
ID	foo
CLOSE_PAREN	
SEMICOLON	

Grammar Example

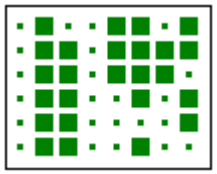


Grammar G_1

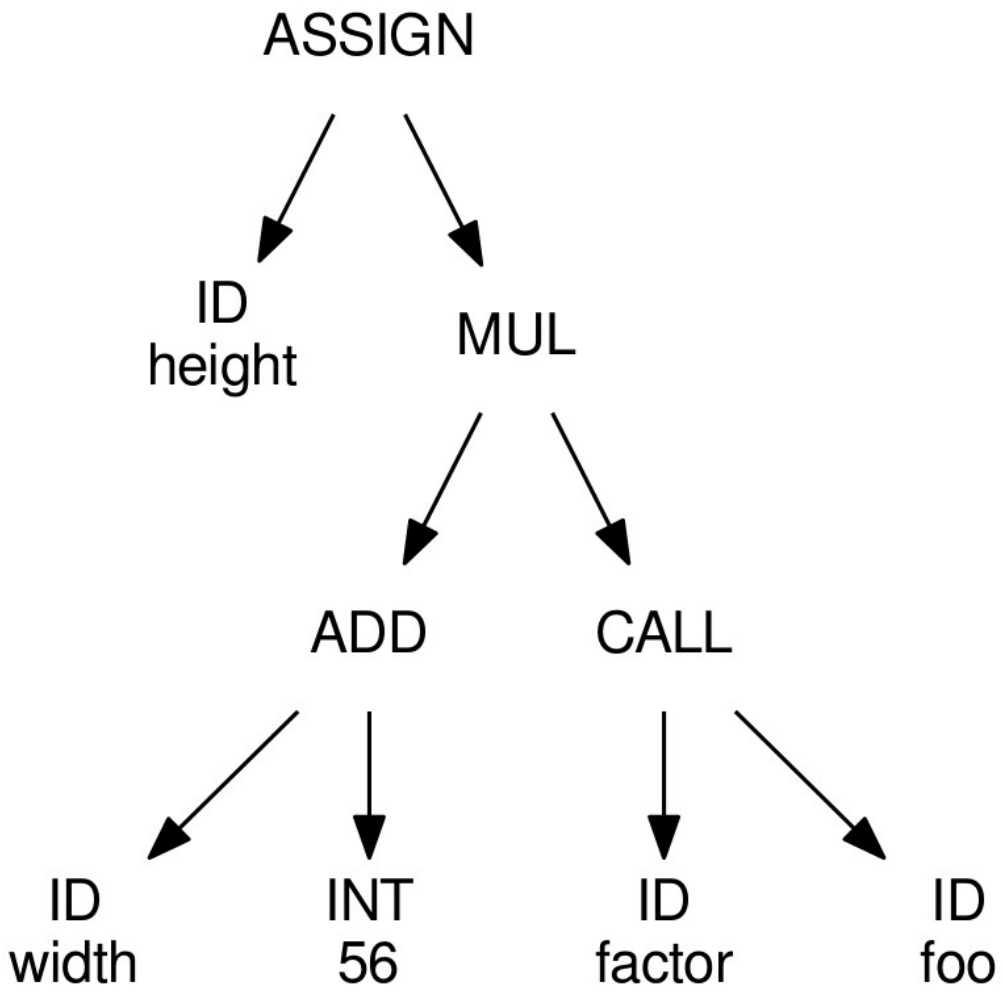
1. expr → expr + expr
2. expr → expr * expr
3. expr → expr = expr
4. expr → id (expr)
5. expr → (expr)
6. expr → id
7. expr → int

Thain, D. (2020). Introduction to compilers and language design.

Abstract Syntax Tree (AST) Example

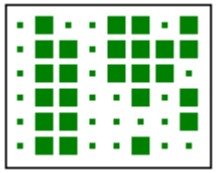


Token	Value
ID	height
ASSIGN	
OPEN_PAREN	
ID	width
ADD	
INT	56
CLOSE_PAREN	
MUL	
ID	factor
OPEN_PAREN	
ID	foo
CLOSE_PAREN	
SEMICOLON	



Thain, D. (2020). Introduction to compilers and language design.

Intermediate Language Example

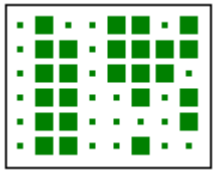


```
LOAD $56     -> r1
LOAD width   -> r2
IADD r1, r2 -> r3
ARG  foo
CALL factor -> r4
IMUL r3, r4 -> r5
STOR r5     -> height
```

Figure 2.4: Example Intermediate Representation

Thain, D. (2020). Introduction to compilers and language design.

Assembly Code Example

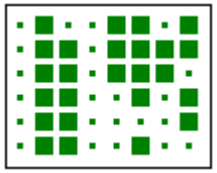


```
MOVQ    width, %rax      # load width into rax
ADDQ    $56, %rax        # add 56 to rax
MOVQ    %rax, -8(%rbp)   # save sum in temporary
MOVQ    foo, %edi        # load foo into arg 0 register
CALL    factor           # invoke factor, result in rax
MOVQ    -8(%rbp), %rbx   # load sum into rbx
IMULQ   %rbx             # multiply rbx by rax
MOVQ    %rax, height     # store result into height
```

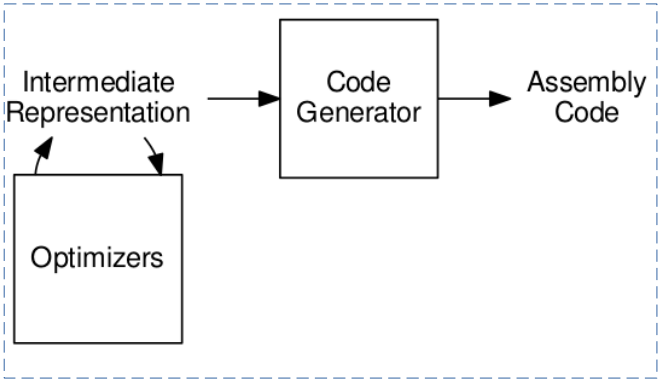
Figure 2.5: Example Assembly Code

Thain, D. (2020). Introduction to compilers and language design.

Optimization and Linking

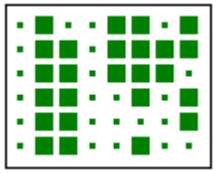


- Optimization may occur at intermediate code stage
  - What might the compiler try to optimize?



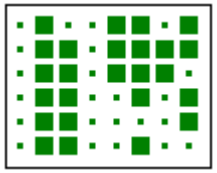
- The code generator takes the intermediate code (and possibly the symbol table) to create the assembly code for the particular architecture on which the code is compiled
- Assembly code is linked with other static libraries to create machine code (executable image, program)
- When run, the executable image (program) must be linked to runtime libraries and allocated resources by the OS (memory, CPU time)

Interpretation



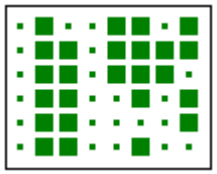
- Program is interpreted at runtime with no (direct) conversion (compilation) to machine language
  - Uses an interpreter to execute the program
    - Conceptually an interpreter is a computer implemented in software (a virtual machine)
    - The interpreter is a compiled program
- The high-level process of interpretation overlaps that of compilation but is a fetch/execute process
- Due to the overhead of runtime interpretation, such languages tend to result in slower execution of a program as compared to an equivalent solution written in a compiled language

Hybrid Compilation/Interpretation

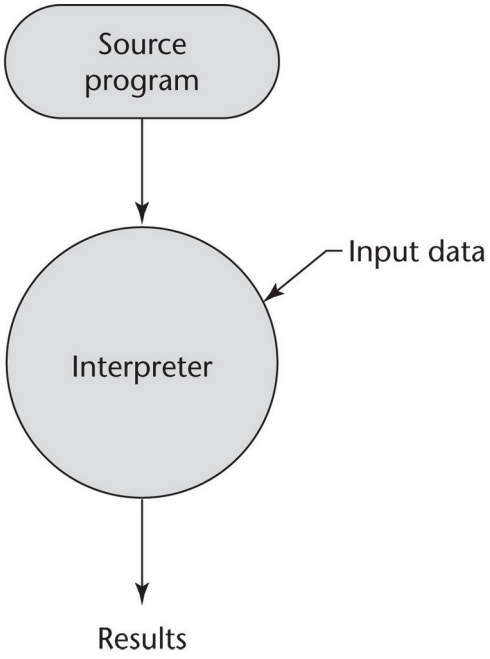


- Pros/Cons of Compilers and Interpreters
  - Compilers result in a more efficient program execution but result is tied to one CPU
  - Interpreters result in a portable program but sluggish runtime behavior
- As interpreter (virtual machine) design became more sophisticated, a middle ground was found
  - Compiler outputs machine language targeting virtual machine (VM) and JIT compiler
  - Any CPU for which the VM is compiled can be used to run (interpret) the compiled program
- Java Runtime Environment (JRE) and .NET Common Language Runtime (.NET CLR) do this

Interpreter and Hybrid Depictions

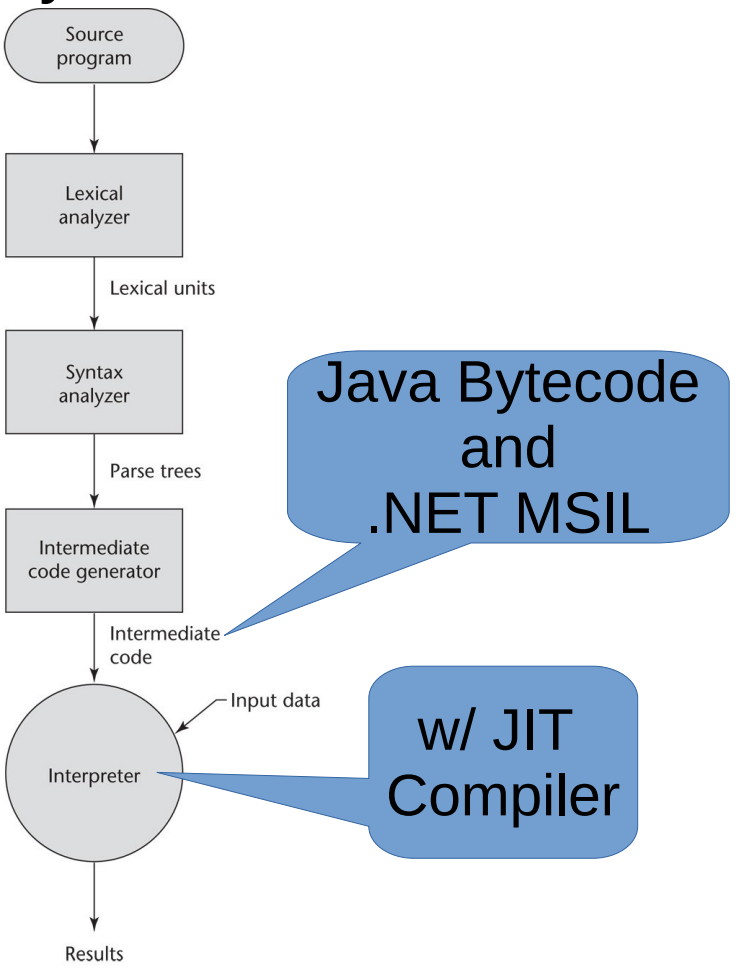


Interpreter



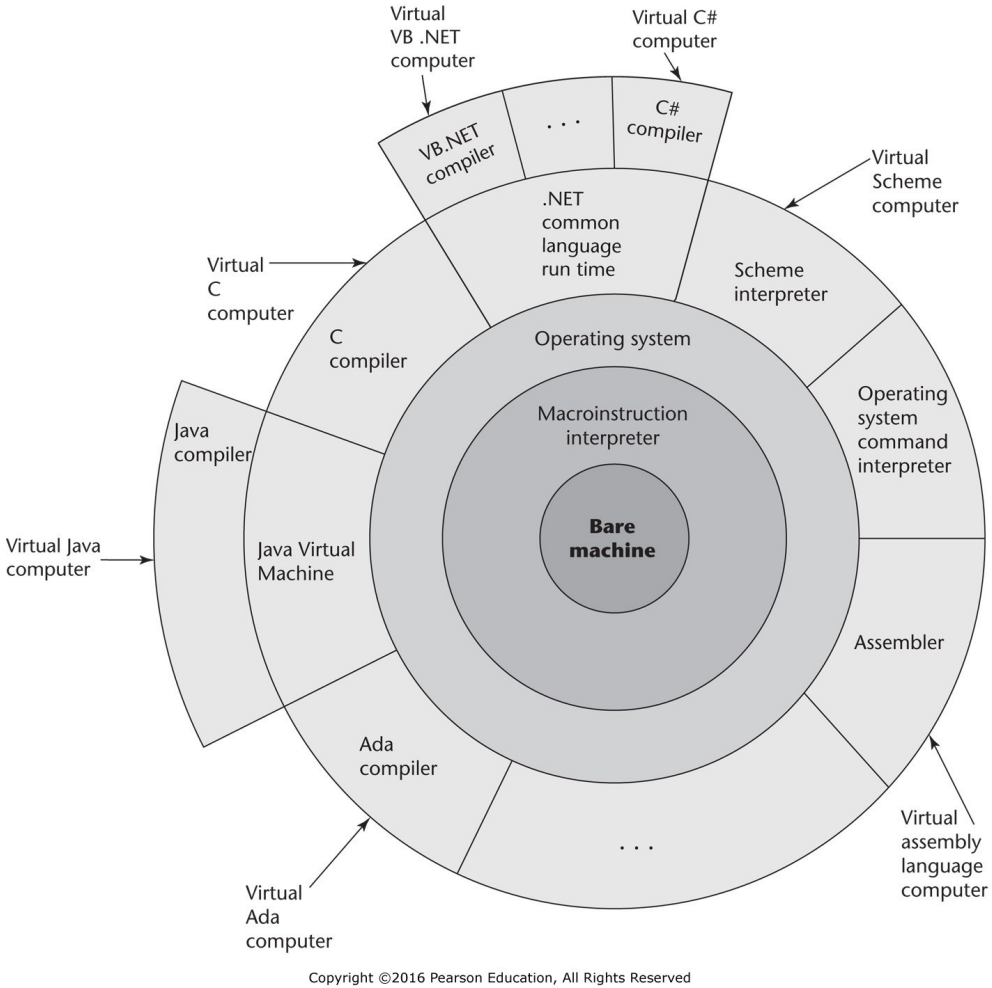
Copyright ©2016 Pearson Education, All Rights Reserved

Hybrid



Sebesta, Robert W., Programming Languages 11/E

Layered Interfaces in Typical Computer



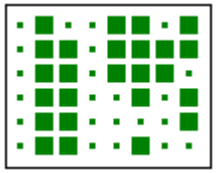
Copyright ©2016 Pearson Education, All Rights Reserved

Could an interpreter for a compiled language be created and, vice-versa, could a compiler for an interpreted language be created?

Put another way, must C be compiled, must Java be a hybrid and must Python be interpreted?

Sebesta, Robert W., Programming Languages 11/E, Figure 1.2, Page 23

Program Editors



- Most languages' syntax are expressed using basic text and common punctuation marks
  - To write programs you use a text editor
- Since languages have a constrained set of keywords and constructs, specialized editors can be used to assist the programmer with repetitive work
  - Integrated Development Environments (IDE)
    - Combine language-aware text editor, syntax hinting and highlighting, integrated interpretation and compilation, syntax error highlighting, debugging facilities, build and deployment support…
    - Eclipse, Visual Studio, VS Code, IDLE...

Homework

- Friday (9/18):
  - Compilers book Chapter 3
  - LLM: AI Coding Assistants…
- Sunday (9/20):
  - Journal entry for week 2
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)

